

Lesson 7 — Scenes, Properly

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

Booleans multiply

4 booleans = **16** states. About 4 are legal. All four false = a game that draws nothing and responds to nothing, with a clean console.

```
let scene = "title";    // one variable, one value
```

Make illegal states impossible to represent, not merely checked for.

A scene

```
const playScene = {
  name: "play",
  enter() { this.score = 0; /* ALL setup */ },
  update(dt) { /* ... */ },
  draw(ctx) { /* ... */ },
  exit() { /* stop sounds */ }
};
```

Setup goes in `enter`, or playing a second time starts where the first one ended.

The loop, for ever

```
Scenes.update(dt);
Scenes.draw(ctx);
```

The stack IS pause

```
update(dt) {
  this.current.update(dt);    // TOP ONLY
}
draw(ctx) {
  for (const s of this.stack)
    s.draw(ctx);              // ALL OF THEM
}
```

- update top only → the game below is frozen
- draw them all → the game below is still visible

```
go(s)    // replace everything
push(s)  // on top, keeping what is below
pop()    // remove the top
```

Just-pressed

```
wasPressed(k) {
  return keys[k] && !previous[k];
}
// at the END of every frame:
previous = Object.assign({}, keys);
```

`keys["p"]` pauses and unpauses thirty times a second.

Do not switch scene mid-update

```
if (dead) { Scenes.go(over); return; }    // return!
```

or defer the change to the end of the frame.

A fade is a scene with a timer

```
draw(ctx) {
  const a = ease(Math.min(1, this.t / TIME));
  ctx.fillStyle = "rgba(0,0,0," + a + ")";
  ctx.fillRect(0, 0, W, H);
}
```

Section A — Recall

A1. Five booleans describe your game's mode. How many combinations are there? Give one illegal combination and say what the player would see. [3 marks]

A2. State the principle about illegal states in your own words, and say how one variable achieves it. [2 marks]

A3. Why does setup belong in `enter` rather than at the top of the file? What is the symptom if it does not? [3 marks]

A4. Write out the two rules of the scene stack, and say what each one gives you. [3 marks]

A5. Why does pausing need *just-pressed* rather than *is held*? [2 marks]

A6. Why must only the top scene read input, and why does that happen by itself? [2 marks]

A7. What is the hazard in calling `Scenes.go()` halfway through `update`, and what is it the same hazard as? [2 marks]

Section B — Predict the output

B1. The stack is `[play, pause]`. Which scenes have `update` called, and which have `draw` called? What does the player see? [3 marks]

B2. `update` is changed to run every scene in the stack. The stack is `[play, pause]`. Describe exactly what the player now experiences. [3 marks]

B3. `draw` is changed to run only the top scene. Stack is `[play, pause]`, and the pause scene draws a translucent black rectangle and the word PAUSED. What appears? [3 marks]

B4. The player holds `P` for one second at 60 fps, using `if (keys["p"]) togglePause()`. How many times does the state change, and is the game paused at the end? Explain. [4 marks]

B5. `FADE_TIME` is 0.5 s. Give the alpha at $t = 0, 0.25$ and 0.6 for a linear fade. Why is the last one worth clamping? [3 marks]

Section C — Find and fix the bug

C1. Playing a second time, the player starts dead with the previous score still showing. [3 marks]

```
const playScene = {
  score: 0,
  lives: 3,
  enter: function () {},
  update: function (dt) { /* ... */ }
};
```

C2. The pause menu appears, the game behind it is visible, and the game is still running underneath. Name the bug and write the fix. [4 marks]

```
update: function (dt) {
  for (let i = 0; i < this.stack.length; i++) {
    this.stack[i].update(dt);
  }
}
```

C3. The console says `Cannot read properties of undefined (reading 'blink')` the moment the title screen appears. [4 marks]

```
const titleScene = {
  enter: function () { this.blink = 0; },
  update: function (dt) { this.blink += dt; }
};
// in the loop:
const u = Scenes.current.update;
u(dt);
```

C4. On the frame the player dies, the console reports an error about `player` being `null`, and one frame of the old scene flashes. [4 marks]

```
update: function (dt) {
  movePlayer(dt);
  if (player.health <= 0) { Scenes.go(gameOverScene); }
  moveEnemies(dt);
  checkCollisions();
}
```

C5. `wasPressed` returns true on every frame the key is held, not just the first. [3 marks]

```
const Input = {
  keys: {}, previous: {},
  wasPressed: function (k) { return this.keys[k] && !this.previous[k]; }
};
// the loop calls Scenes.update(dt) and Scenes.draw(ctx), and nothing else
```

Section D — Build it

Use the game you built in lessons 5 and 6.

- ☐ **Checkpoint 1.** Add `wasPressed` to your input code, with the `previous` copy at the end of every frame. Test it by counting presses on screen — hold the key and check the number goes up by exactly one.
- ☐ **Checkpoint 2.** Turn your existing game into a `playScene` object with `enter`, `update`, `draw` and `exit`. Move *all* its setup into `enter`. Your loop should now be two lines.
- ☐ **Checkpoint 3.** Add a title scene and a game over scene. Get the full circle working: title → play → game over → title. Play it twice in a row and check the second game is identical to the first.

- ☐ **Checkpoint 4.** Build the stack: `go`, `push`, `pop`, and the two loops. Add a pause scene that draws a translucent rectangle and the word **PAUSED**. It should be about six lines and the game should be visible, frozen, behind it.
- ☐ **Checkpoint 5.** Draw the stack on screen as a list of scene names while you play. Keep it — it is the best debugging aid in this lesson.
- ☐ **Checkpoint 6.** Add a settings scene you can push *on top of* the pause scene, and pop back from. Three scenes deep, with the game still visible at the bottom.
- ☐ **Checkpoint 7.** Add a fade transition between title and play, using an easing curve rather than a straight line.
- ☐ **Checkpoint 8.** Make a scene change mid-update on purpose, without returning, and write down what happens. Then fix it the way you prefer and say why.

IF THIS IS UNDEFINED

You have pulled a method off its object (`const u = scene.update`) and lost the connection. Call it as `scene.update(dt)`, or use `scene.update.call(scene, dt)` . The solutions also show a version built from closures, with no `this` anywhere.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. Where should the score live: in the play scene, in a global, or passed from scene to scene? The game over screen needs to show it. What does each choice cost?

E2. The game over screen ignores input for half a second after it appears. Why would anybody do that deliberately? Think about *how* the player arrived there.

E3. A settings screen reachable from both the title and the pause menu, which returns to whichever it came from. Does the stack solve this completely, or is something left to decide?

E4. The hard one. A level transition: the player touches a door, the screen fades out, the next level loads, it fades in, and the score and health carry over. Which parts are scenes, which are data, and **where does the data live** while no scene owns it? Draw it before you write it.

Stretch goals

1. `Scenes.fadeTo(nextScene, 0.4)` as a proper transition manager.
2. A loading scene that waits two seconds and then replaces itself. Where does what it loaded go?
3. A dialogue box over the world: text a character at a time, space to advance, the world still visible. One `push`.
4. Draw the stack on screen, then try to get it into a state you did not intend.
5. The level transition (E4), for real.